ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №3

«Ітератори, генератори та потокова обробка даних»

з курсу «Професійний Python»
Варіант №13 — «Система обліку комп'ютерної техніки»

Виконав: Капарник Дмитро Андрійович

Група: ФЕП-32

Перевірив: доцент Куньо Іван Михайлович

Дата: 18.10.2026

Львів — 2026

1. Мета роботи

Опанувати ітераторний протокол, генератори, ліниві конвеєри та вимірювання пам’яті під час обробки великих CSV.

2. Індивідуальний варіант і постановка задачі

Варіант №13 — система обліку комп’ютерної техніки. Створити потокову валідацію записів, фільтр виробника/типу, пошук номера, середню/загальну вартість, максимум, лічильники та batching.

Наскрізний проєкт продовжує ЛР №1. Базові поля: інвентарний номер, тип, виробник, модель і вартість. Збережено пошук, фільтр виробника, середню/загальну вартість і визначення найдорожчого пристрою.

3. Структура проєкту та створені модулі

```
inventory-service/
  pyproject.toml, README.md, .gitignore
  src/computer_inventory/
    models.py
    streaming.py
    analytics.py
    main.py
  src/inventory_service/  # фінальний пакет запуску
  tests/
  scripts/
  docs/GITHUB.md
```

4. Вхідні дані та схема конвеєра

```
inventory_number,type,manufacturer,model,cost
INV-0000001,Computer,Dell,Model-0,1000.25
INV-0000002,Printer,HP,Model-1,1001.25
```

Набори формуються детерміновано: тип чергується, виробник обирається циклічно з Dell/HP/Lenovo, ціна дорівнює 1000 + i mod 90000 з копійками .25. Для порівняння використано 10 000, 100 000 та 500 000 записів; для early termination — 1 000 000.

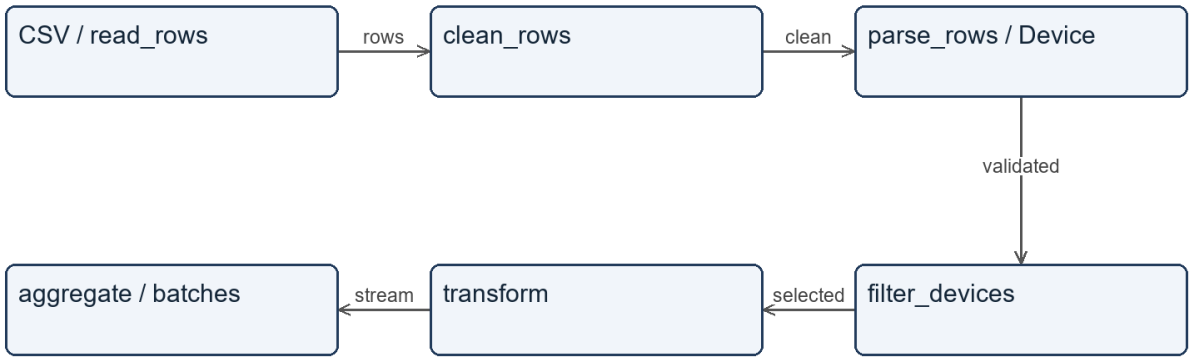


Рисунок 1 — Схема лінивого потокового конвеєра

5. Ітератор, генератори та itertools

Iterable дозволяє отримати ітератор; iterator зберігає стан і повертає наступний елемент через __next__. Власний InventoryNumbers використовує count. Функції read_rows, clean_rows, parse_rows, filter_devices, transform, pipeline і batches є генераторами. Використано count, islice та chain; multi_file об’єднує потоки кількох файлів. Вираз (pipeline(path) for path in paths) є generator expression.

```
class InventoryNumbers(Iterator[str]):
    def __init__(self, prefix: str = "INV") -> None:
        self.prefix = prefix
        self._numbers = count(1)

    def __iter__(self) -> "InventoryNumbers":
        return self

    def __next__(self) -> str:
        return f"{self.prefix}-{next(self._numbers):07d}"

def read_rows(path: Path) -> Iterator[dict[str, str]]:
    with path.open(encoding="utf-8", newline="") as stream:
        yield from csv.DictReader(stream)

def clean_rows(rows: Iterable[dict[str, str]]) -> Iterator[dict[str, str]]:
    for row in rows:
        if None in row or any(value is None for value in row.values()):
            raise ValueError("Malformed CSV row")
        yield {key.strip(): value.strip() for key, value in row.items()}

def parse_rows(rows: Iterable[dict[str, str]]) -> Iterator[Device]:
    for row in rows:
        yield Device(
            row["inventory_number"],
            row["type"],
            row["manufacturer"],
            row["model"],
            Decimal(row["cost"]),
```

```
        )
```

6. Фільтрація, перетворення, агрегація та batching

```
def filter_devices(
    devices: Iterable[Device], manufacturer: str = "", device_type: str = ""
) -> Iterator[Device]:
    for device in devices:
        if manufacturer and device.manufacturer.casefold() != manufacturer.casefold():
            continue
        if device_type and device.device_type.casefold() != device_type.casefold():
            continue
        yield device

def transform(devices: Iterable[Device]) -> Iterator[Device]:
    """Normalize outer whitespace through the validated immutable model."""
    for device in devices:
        yield Device(
            device.inventory_number,
            device.device_type,
            device.manufacturer,
            device.model,
            device.cost,
        )

def pipeline(
    path: Path, manufacturer: str = "", device_type: str = ""
) -> Iterator[Device]:
    yield from transform(
        filter_devices(
            parse_rows(clean_rows(read_rows(path))), manufacturer, device_type
        )
    )

def multi_file(paths: Iterable[Path]) -> Iterator[Device]:
    yield from chain.from_iterable(pipeline(path) for path in paths)

def batches(items: Iterable[Device], size: int = 1000) -> Iterator[tuple[Device, ...]]:
    if size < 1:
        raise ValueError("Batch size must be positive")
    iterator = iter(items)
    while batch := tuple(islice(iterator, size)):
        yield batch

@dataclass
class StreamSummary:
    count: int = 0
    total: Decimal = Decimal("0.00")
    most_expensive: Device | None = None
    types: Counter[str] = field(default_factory=Counter)
    manufacturers: Counter[str] = field(default_factory=Counter)

    @property
    def average(self) -> Decimal:
```

```
        return (
            (self.total / self.count).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
            if self.count
            else Decimal("0.00")
        )

def aggregate(devices: Iterable[Device]) -> StreamSummary:
    result = StreamSummary()
    for device in devices:
        result.count += 1
        result.total += device.cost
        result.types[device.device_type] += 1
        result.manufacturers[device.manufacturer] += 1
        if result.most_expensive is None or device.cost > result.most_expensive.cost:
            result.most_expensive = device
    return result
```

Пошук номера реалізується next((d for d in stream if d.inventory_number == number), None). При ранній зупинці генератор закривається через contextlib.closing, щоб не залишати відкритий файл. Агрегатор зберігає лише суму, кількість, максимум і Counter типів/виробників; для фіксованої кількості категорій його додаткова пам’ять майже стала.

7. Порівняння eager та lazy

```
def measure(path: Path, eager: bool) -> dict[str, float]:
    tracemalloc.start()
    start = perf_counter()
    if eager:
        items = list(pipeline(path))
        first = perf_counter() - start
        result = aggregate(items)
    else:
        with closing(pipeline(path)) as items:
            next(items, None)
            first = perf_counter() - start
        result = aggregate(pipeline(path))
    elapsed = perf_counter() - start
    peak = tracemalloc.get_traced_memory()[1] / 1024**2
    tracemalloc.stop()
    return {
        "elapsed_s": elapsed,
        "peak_MiB": peak,
        "first_s": first,
        "count": result.count,
        "total": str(result.total),
    }
```

Записів	**Eager, с**	**Lazy, с**	**Eager, МіБ**	**Lazy, МіБ**
10000	0.2732	0.2562	3.955	0.052
100000	3.2414	3.4502	39.142	0.053
500000	16.5584	13.7671	195.703	0.053

Записів	Eager, с	Lazy, с	Eager, МіБ	Lazy, МіБ

Записів	Перший eager, с	Перший lazy, с
10000	0.254076	0.000560
100000	2.853235	0.000786
500000	15.355902	0.000632

На 1 000 000 записів перший Dell (INV-0000001) знайдено ліниво за 0.006343 с; варіант із попереднім формуванням усього списку збігів — за 9.602374 с. Результати збігаються. Вимірювання tracemalloc показує пік відстежуваних Python-алокацій, а не повний RSS. У lazy-вимірюванні перший результат перевіряється окремим коротким читанням, після чого повний прохід починається заново.

Матеріалізація великих списків збільшує пікову пам'ять. Генератор не гарантує скорочення загального часу; його головні переваги — обмежений стан і ранній результат. Перетворення list(pipeline(...)) скасовує цю перевагу. У ЛР3 валідація перевіряє кожен запис, а глобальна унікальність номерів із множиною seen додається в імпортері ЛР5.

Код і історія лабораторних: https://github.com/dimakaparnyk/professional-python-labs. Для відповідного етапу використовується тег lab-03, фінальна інтегрована реалізація — main (перевірений CI commit 2e3fb3d).

8. Відповіді на контрольні запитання

1. Що таке iterable?

Об'єкт, з якого iter() може отримати ітератор.

2. Що таке iterator?

Об'єкт із __iter__() та __next__(), який послідовно повертає елементи.

3. Чим iterable відрізняється від iterator?

Iterable описує можливість перебору; iterator зберігає поточний стан конкретного перебору.

4. Для чого використовується iter()?

Для отримання ітератора з перебираного об'єкта.

5. Для чого використовується next()?

Для отримання наступного елемента або заданого значення за замовчуванням після вичерпання.

6. Що робить __iter__()?

Повертає ітератор для об'єкта; ітератор зазвичай повертає самого себе.

7. Що робить __next__()?

Повертає наступний елемент або піднімає StopIteration.

8. Що таке StopIteration?

Виняток, який сигналізує нормальне завершення ітерації.

9. Як цикл for використовує iterator protocol?

Викликає iter(), а потім повторює next() до StopIteration.

10. Чому iterator часто називають one-shot object?

Зазвичай він проходить послідовність один раз і після вичерпання не починає її знову.

11. Чому iter(iterator) is iterator?

Тому що ітератор уже містить власний стан перебору й не потребує нового ітератора.

12. Чи можна повторно використати exhausted generator?

Ні; потрібно заново викликати генераторну функцію та створити новий генератор.

13. Що таке generator function?

Функція, яка містить yield і під час виклику повертає генератор.

14. Яку роль виконує yield?

Видає поточне значення та призупиняє виконання зі збереженням стану.

15. Чим yield відрізняється від return?

yield призупиняє функцію, а return завершує її; у генераторі return завершує ітерацію.

16. Коли фактично починає виконуватися generator function?

Під час першого next(), send() або споживання в циклі.

17. Що зберігає generator між викликами next()?

Локальні змінні, позицію виконання та посилання на потрібні ресурси.

18. Що таке generator expression?

Компактний вираз створення лінивого генератора, наприклад (d.cost for d in devices).

19. Чим generator expression відрізняється від list comprehension?

Він не будує весь список одразу й зазвичай споживається одноразово.

20. Що таке lazy evaluation?

Обчислення значень лише тоді, коли вони потрібні споживачу.

21. Що таке eager evaluation?

Негайне обчислення та зазвичай матеріалізація всього результату.

22. Які переваги lazy evaluation?

Менше одночасно збережених даних, швидший перший результат і дострокова зупинка.

23. Які недоліки lazy evaluation?

Одноразовість, відкладені помилки, складніший контроль ресурсів та накладні витрати ітерації.

24. Що таке streaming?

Послідовна обробка невеликих частин даних без повного завантаження джерела.

25. Чому streaming важливий для великих files?

Дає змогу обробляти файл, більший за доступну оперативну пам’ять.

26. Чому read() може бути проблемою для multi-GB file?

read() без обмеження може створити багатогігабайтний рядок і вичерпати пам’ять.

27. Чому file object є iterable?

Текстовий файл підтримує ітераторний протокол і видає наступні рядки.

28. Для чого використовується yield from?

Для делегування видачі значень іншому iterable або вкладеному генератору.

29. Що таке itertools?

Модуль стандартної бібліотеки з ефективними інструментами композиції ітераторів.

30. Для чого itertools.count()?

Створює необмежену арифметичну послідовність, корисну для номерів записів.

31. Чому count() потрібно обмежувати?

Без межі або дострокового виходу споживання нескінченної послідовності не завершиться.

32. Для чого islice()?

Для лінивого отримання зрізу або перших k елементів ітератора.

33. Для чого chain()?

Для послідовного об’єднання кількох iterable без побудови спільного списку.

34. Як працює takewhile()?

Видає елементи, доки предикат істинний; перший невідповідний елемент зупиняє перебір.

35. Як працює dropwhile()?

Пропускає початкові елементи, доки предикат істинний, потім видає решту.

36. Для чого accumulate()?

Для послідовних проміжних підсумків або іншої накопичувальної операції.

37. Для чого pairwise()?

Для перебору сусідніх пар елементів.

38. Як працює groupby()?

Групує лише суміжні елементи з однаковим ключем.

39. Чому перед groupby() часто потрібне sorting?

Щоб усі елементи з однаковим ключем стали суміжними та утворили одну групу.

40. Що робить product()?

Створює декартів добуток вхідних послідовностей.

41. Чим combinations() відрізняється від permutations()?

combinations не враховує порядок вибору, permutations враховує різні порядки.

42. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?

Кількість результатів може бути величезною; лінивість зменшує пам’ять, але не загальну кількість обчислень.

43. Що таке batching?

Поділ потоку на обмежені пакети елементів.

44. Які переваги batching?

Контроль пам’яті, менше накладних витрат на операцію та зручна пакетна взаємодія з БД.

45. Що таке lazy pipeline?

Ланцюжок ітераторів, де кожен етап обчислює елементи лише за запитом наступного.

46. Які stages може містити pipeline?

Читання, очищення, розбір, валідація, фільтрація, перетворення та агрегація.

47. Що є source у streaming pipeline?

Файл, мережевий потік, генератор або інший початковий постачальник записів.

48. Що є transformation?

Перетворення одного запису на інше представлення або нормалізація його значень.

49. Що є sink?

Кінцевий споживач: агрегатор, файл експорту, база даних або виведення.

50. Чи гарантує generator менший total execution time?

Ні; перевага може бути лише в пам’яті або швидкості першого результату.

51. Яка основна memory advantage generator?

Він не зберігає всі видані об’єкти одночасно; пам’ять залежить від стану етапів.

52. Що таке time to first result?

Час від початку обробки до отримання першого доступного результату.

53. Чому lazy pipeline добре підтримує early termination?

Подальші елементи ще не обчислені, тому споживач може припинити їх запитувати.

54. Чому exception у generator може виникати із затримкою?

Код генератора виконується під час споживання, а не створення об’єкта.

55. Чому list(generator) може бути проблемою?

Перетворення у list матеріалізує весь потік і скасовує головну перевагу щодо пам’яті.

56. Що трапиться при list(count())?

Виклик не завершиться нормально й зростатиме споживання пам’яті до помилки або переривання.

57. Як generator може впливати на lifecycle file resource?

Відкритий у генераторі файл лишається відкритим до завершення або явного закриття генератора.

58. Як тестувати невеликий generator?

Порівняти list(generator) з очікуваною короткою послідовністю та перевірити винятки.

59. Як тестувати infinite generator?

Обмежити його через islice та перевірити скінченний префікс.

60. Коли краще використовувати list, а коли generator?

list зручний для повторного доступу й індексації, generator — для одного проходу великого потоку.

9. Висновок

Побудовано конвеєр із генераторним читанням, очищенням, розбором, фільтрацією, перетворенням та агрегацією. Доведено збіг підсумків eager/lazy на великих наборах. Потокова обробка суттєво зменшила одночасно збережені об’єкти та час до першого результату.